

Solution - Implement a Circular Singly Linked List :

```
public class CSLinkedList {
    Node head;
    Node tail;
    int length;

    public CSLinkedList() {
        this.head = null;
        this.tail = null;
        this.length = 0;
    }

    public void append(int value) {
        Node newNode = new Node(value);
        if (this.length == 0) {
            this.head = newNode;
            this.tail = newNode;
            newNode.next = newNode;
        } else {
            this.tail.next = newNode;
            newNode.next = this.head;
            this.tail = newNode;
        }
        this.length++;
    }

    public void prepend(int value) {
        Node newNode = new Node(value);
        if (this.length == 0) {
            this.head = newNode;
            this.tail = newNode;
            newNode.next = newNode;
        } else {
            newNode.next = this.head;
        }
    }
}
```

```

        this.head = newNode;
        this.tail.next = newNode;
    }
    this.length++;
}

@Override
public String toString() {
    if (this.length == 0) {
        return "";
    }

    Node tempNode = this.head;
    StringBuilder result = new
StringBuilder();

    do {
        result.append(tempNode.value);
        tempNode = tempNode.next;
        if (tempNode != this.head) {
            result.append(" -> ");
        }
    } while (tempNode != this.head);

    return result.toString();
}
}

```

This Java code defines a class **CSLinkedList** that represents a Circular Singly Linked List. Each node in the list is represented by an instance of the **Node** class, which is not fully shown here but is presumed to have an **int value** and a **Node next** to point to the next node. Here's a line-by-line explanation:

Class Definition and Fields

- `public class CSLinkedList {` - Defines a public class named `CSLinkedList`.
- `Node head;` - Declares a `Node` variable `head` to store the reference to the first node in the list.
- `Node tail;` - Declares a `Node` variable `tail` to store the reference to the last node in the list.
- `int length;` - Declares an integer `length` to keep track of the number of nodes in the list.

Constructor

- `public CSLinkedList() {` - Defines the constructor for the `CSLinkedList` class.
- `this.head = null;` - Initializes `head` to `null`, indicating that the list is initially empty.
- `this.tail = null;` - Initializes `tail` to `null`, indicating that the list is initially empty.
- `this.length = 0;` - Initializes the `length` of the list to 0.

append Method

- `public void append(int value) {` - Defines a method to append a new node with a given value to the end of the list.
- `Node newNode = new Node(value);` - Creates a new `Node` instance with the provided value.
- `if (this.length == 0) {` - Checks if the list is empty.
- `this.head = newNode;` - Sets `head` to the new node if the list was empty.
- `this.tail = newNode;` - Sets `tail` to the new node if the list was empty.
- `newNode.next = newNode;` - Points `newNode.next` to itself, making the list circular.
- `} else {` - If the list is not empty.

- `this.tail.next = newNode;` - Links the current tail node to the new node.
- `newNode.next = this.head;` - Sets the new node's next pointer to the head, maintaining circularity.
- `this.tail = newNode;` - Updates the tail to be the new node.
- `}` - Ends the conditional block.
- `this.length++;` - Increments the length of the list.

prepend Method

- `public void prepend(int value) {` - Defines a method to prepend a new node with a given value to the start of the list.
- Similar steps to `append`, but this method inserts the new node at the beginning and updates the `head` and the `tail.next` accordingly.

toString Method

- `@Override public String toString() {` - Overrides the `toString` method to provide a string representation of the list.
- Checks if the list is empty and returns an empty string if so.
- Initializes a temporary node to iterate through the list and a `StringBuilder` to build the string.
- Uses a do-while loop to traverse the list once, appending each node's value to the `StringBuilder`.
- Returns the string representation of the list.

Time and Space Complexity

- **append and prepend Methods:** Both methods have a time complexity of $O(1)$ because they add a node either at the

beginning or the end of the list, requiring a constant number of operations regardless of the list size.

- **toString Method:** This method has a time complexity of $O(n)$, where n is the number of nodes in the list. It iterates through each node once to build the string representation.
- **Space Complexity:** The space complexity for the list is $O(n)$, where n is the number of nodes, accounting for the space used by each node in the list.

Solution - Delete a Node from a Circular Singly Linked List :

```
public class CircularLinkedList {  
  
    Node head;  
    Node tail;  
    int length;  
  
    public CircularLinkedList() {  
        this.head = null;  
        this.tail = null;  
        this.length = 0;  
    }  
  
    public void append(int value) {  
        Node newNode = new Node(value);  
        if (this.length == 0) {  
            this.head = newNode;  
            this.tail = newNode;  
            newNode.next = newNode; // Pointing  
to itself to make the list circular  
        }  
    }  
}
```

```

        } else {
            this.tail.next = newNode;
            newNode.next = this.head;
            this.tail = newNode;
        }
        this.length++;
    }

    @Override
    public String toString() {
        if (this.length == 0) {
            return "";
        }

        Node tempNode = this.head;
        StringBuilder result = new
StringBuilder();

        do {
            result.append(tempNode.value);
            tempNode = tempNode.next;
            if (tempNode != this.head) {
                result.append(" -> ");
            }
        } while (tempNode != this.head);

        return result.toString();
    }

    public boolean deleteByValue(int value) {
        if (this.length == 0) {
            return false;
        }
    }

```

```

        if (this.head == this.tail &&
this.head.value == value) {
            this.head = null;
            this.tail = null;
            this.length = 0;
            return true;
        }

Node current = this.head;
Node prev = null;

do {
    if (current.value == value) {
        if (current == this.head) {
            this.head = this.head.next;
            this.tail.next = this.head;
        } else if (current ==
this.tail) {
            prev.next = this.head;
            this.tail = prev;
        } else {
            prev.next = current.next;
        }

        this.length--;
        return true;
    }

    prev = current;
    current = current.next;
} while (current != this.head);

return false;
}
}

```

Let's break down the method line by line:

1. **public boolean deleteByValue(int value) {** - This method signature declares a public method that returns a boolean indicating whether a node was successfully deleted. It accepts an **int value** as the parameter, which is the value to search for and delete.
2. **if (this.length == 0) {** - Checks if the list is empty. If **length** is 0, there are no nodes to delete.
3. **return false;** - If the list is empty, the method returns **false**, indicating that no deletion occurred.
4. **if (this.head == this.tail && this.head.value == value) {** - Checks if there's only one node in the list (head equals tail) and if that node's value matches the target value.
5. **this.head = null;** - If the single node matches the target value, set **head** to **null**.
6. **this.tail = null;** - Also set **tail** to **null**, effectively removing the node and leaving an empty list.
7. **this.length = 0;** - Set the length of the list to 0.
8. **return true;** - Returns **true**, indicating that the node was successfully deleted.
9. **Node current = this.head;** - Initializes a **current** pointer to start traversal from the head of the list.
10. **Node prev = null;** - Initializes a **prev** pointer to **null**. This will keep track of the node immediately preceding **current**.
11. **do {** - Starts a do-while loop to traverse the list.
12. **if (current.value == value) {** - Checks if the **current** node's value matches the target value.
13. **if (current == this.head) {** - If the node to delete is the head, additional steps are taken to update the head.

14. `this.head = this.head.next;` - Sets the head to the next node in the list.
15. `this.tail.next = this.head;` - Updates the tail's next pointer to the new head, maintaining the circular nature.
16. `} else if (current == this.tail) {` - Checks if the node to delete is the tail.
17. `prev.next = this.head;` - Bypasses the tail by setting the previous node's next to the head.
18. `this.tail = prev;` - Sets the tail to the previous node.
19. `} else {` - If the node to delete is neither the head nor the tail (i.e., a middle node).
20. `prev.next = current.next;` - Bypasses the current node by linking the previous node directly to the next node.
21. `}` - Closes the if-else block.
22. `this.length--;` - Decreases the length of the list by 1, as a node has been removed.
23. `return true;` - Returns `true`, indicating successful deletion.
24. `}` - Closes the if block that checks for value match.
25. `prev = current;` - Moves the `prev` pointer up to the current node.
26. `current = current.next;` - Advances the `current` pointer to the next node.
27. `} while (current != this.head);` - Continues looping until it reaches back to the head, indicating a full traversal of the circular list.
28. `return false;` - If no node with the target value is found after traversing the list, returns `false`.

Time Complexity:

- The time complexity is $O(n)$, where n is the number of nodes in the list. In the worst case, the method might need to traverse the entire list to find the node to delete.

Space Complexity:

- The space complexity is $O(1)$ since the method uses a constant amount of space regardless of the size of the list.

Solution to Count the Number of Nodes :

```
public class CSLinkedList {
    Node head;
    Node tail;
    int length;

    public CSLinkedList() {
        this.head = null;
        this.tail = null;
        this.length = 0;
    }

    // Other methods...

    // Method to append a node at the end of
    the list
    public void append(int value) {
        Node newNode = new Node(value);
        if (this.length == 0) {
            this.head = newNode;
            this.tail = newNode;
            newNode.next = newNode; // Pointing
to itself to make the list circular
        } else {
            this.tail.next = newNode;
            newNode.next = this.head;
        }
    }
}
```

```

        this.tail = newNode;
    }
    this.length++;
}

// Method to convert the list into a string
representation
@Override
public String toString() {
    if (this.length == 0) {
        return "";
    }

    Node tempNode = this.head;
    StringBuilder result = new
StringBuilder();

    do {
        result.append(tempNode.value);
        tempNode = tempNode.next;
        if (tempNode != this.head) {
            result.append(" -> ");
        }
    } while (tempNode != this.head);

    return result.toString();
}

public int countNodes() {
    int count = 0;
    Node temp = this.head;
    if (temp == null) {
        return 0; // Return 0 immediately
if the list is empty
    }
}

```

```

        do {
            count++; // Increment the count
for each node
            temp = temp.next; // Move to the
next node
        } while (temp != this.head); //
Continue until the list cycles back to the
head

        return count; // Return the total
count
    }
}

```

Explanation:

1. **public int countNodes() {** - Defines the **countNodes** method that returns an integer, representing the number of nodes in the list.
2. **int count = 0;** - Initializes a counter variable **count** to 0 to keep track of the number of nodes.
3. **Node temp = this.head;** - Initializes a temporary node **temp** to point to the head of the list for traversal.
4. **if (temp == null) { return 0; }** - Immediately returns 0 if the list is empty (i.e., **head** is **null**).
5. **do {** - Starts a do-while loop which ensures that the body of the loop executes at least once.
6. **count++;** - Increments the **count** variable for each node encountered.
7. **temp = temp.next;** - Moves the **temp** pointer to the next node in the list.
8. **} while (temp != this.head);** - The loop continues until **temp** circles back to the head, which indicates a complete traversal of the circular list.

9. **return count;** - Returns the total count of nodes in the list.

Time Complexity:

- **$O(n)$:** The method has a time complexity of $O(n)$, where n is the number of nodes in the list. This is because, in the worst-case scenario, the method will traverse each node exactly once.

Space Complexity:

- **$O(1)$:** The space complexity is $O(1)$ since the method uses a constant amount of space, with only a few primitive variables, irrespective of the number of nodes in the list.